



Ensemble learning-based adaptive and adversarial techniques for APT attack detection

Nguyen Thanh Tung¹ · Cho Do Xuan² · Vu Thanh Long²

Accepted: 2 May 2025

© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2025

Abstract

Nowadays, Advanced persistent threats (APT) attacks are not only increasing in number but also becoming more sophisticated, posing serious challenges for organizations in detecting and preventing these threats. Some approaches for detecting APT attacks based on network traffic analysis using artificial intelligence have initially brought many positive results. The trend of these studies often seeks to analyze and extract network flow information in network traffic to find unusual behaviors of APT attacks. However, these approaches still have certain disadvantages that limit detection results, including: i) Only process data sequentially, lack flexibility and do not filter and focus on important data; ii) the context in the data has not been defined and evaluated. Although this is important information, it is closely related and interdependent with abnormal signs. To overcome the above limitations, this study introduces an advanced learning ensemble, combining modern techniques such as Adaptive contextual (AC), Selective adversarial generation (SAG), BiLSTM (Bidirectional long short-term memory), and Transformer. The AC technique is used to adapt to different network traffic data, helping to distill important information and adapt itself to data with different numbers of attacks, demonstrating context preservation and detecting anomalous patterns. SAG is applied to create synthetic data to simulate hacker attack data, helping to balance and improve the learning ability of the model. BiLSTM and Transformer are integrated to exploit contextual relationships and long-term dependencies in data, enhancing data processing capabilities based on parallel computing and analysis techniques. Experimental results in the article show that the proposed model is 2–4% more effective than other studies on all measures. The experimental results of the study show that the proposed ACG-BT model has achieved an accuracy of up to 99.45% and an F1-score of 98.35% on a data set of 1,884 IP pairs extracted from more than 1.6 million flows. These figures demonstrate the model's superior performance compared to existing methods, with improvements ranging from 2 to 4% across all metrics.

Keywords APT attack detection · Ensemble learning model · Adaptive contextual · Transformer

1 Introduction

1.1 Overview

APT is a type of targeted cyber attack on agencies, companies, organizations, etc. [1] in which the attacker will create an illegal and long-term presence on the victim's network

system to exploit sensitive data such as employee data, customer data, trade secrets, ... [2, 3]. APT uses many complex techniques that are meticulously and methodically designed to maintain existence in the system for a long time, even when performing illegal actions in the system, it is still difficult to detect [4].

To solve the task of detecting and early warning of APT attacks, some typical methods such as machine learning and deep learning used to analyze network traffic [5–10] have been proposed. In particular, recently, a number of Ensemble Learning models have been researched and published. The APT detection approach based on flow analysis in network traffic becomes an effective direction thanks to its ability to identify unusual behaviors in detail and accurately. Instead of relying solely on signatures or static rules, this method exploits network flow characteristics to detect suspicious

✉ Cho Do Xuan
chodx@ptit.edu.vn

¹ National Institute of Digital Technology and Digital Transformation, Ministry of Information and Communications, Hanoi, Vietnam

² Department of Information Security, Posts and Telecommunications Institute of Technology, Hanoi, Vietnam

behavior patterns, even when APT attacks use evasion techniques such as never-before-seen malware or advanced attack techniques. By monitoring and analyzing flow characteristics such as connection time, packet frequency, latency, and relationships between IP pairs, etc. this method can identify unusual signs that traditional systems often miss.

In addition to studies focusing on network traffic analysis to detect APT attacks, recent literature on security in link recommender systems also provides valuable perspectives on attack and defense techniques that may be relevant or applicable in the context of cyber attack detection. For example, studies such as [11] address interaction-level inference attacks in link recommender systems, while [12] analyzes manipulation attacks targeting image-aware link recommender systems and countermeasures. Similarly, [13] discusses adversarial item promotion attack in image recognition recommender system using guided diffusion technique. A comprehensive survey [14] provides an overview of poisoning attacks and defense methods in recommender systems, emphasizing the need to protect machine learning models from threats. Finally, [15] proposed the PDC-FRS method to preserve privacy in link recommender systems, an idea that could inspire techniques for protecting network data against APT attacks. These studies, although developed in the field of recommender systems, show the potential for applying similar techniques to improve detection and defense against APT threats in networks.

However, these studies only process data sequentially without considering special relationships in the data, creating certain problems such as not filtering and focusing on important data, causing waste of information, the model does not focus on the information that needs to be resolved, and the learned information is noisy. The inability to self-adjust to datasets with different numbers of attack samples, missing the complex relationships of sophisticated attack samples will limit the applicability of the model to diverse datasets or new attack data.

To solve that problem, new proposals have appeared such as combining models to create a larger network model or using more powerful and complex networks such as graph networks [16–19]. Although these studies have opened up new directions in combining models and using graph networks to sketch data relationships, improving analysis capabilities. However, there are still certain limitations including:

- The first is the problem of synthesizing and collecting contextual information adapted to different data: Not collecting and synthesizing important information will cause noise and confusion when building relationships in data, blurring the interaction of the data we need to focus on with other data. In addition, traditional models do not pay attention to the context in surrounding data related to abnormal

signs. Failure to maintain context will miss complex relationships and long-term dependencies, leading to failure to detect subtle abnormal patterns, reducing model accuracy.

- The second is to generate synthetic data to simulate attack data in the network: If there is no method to simulate the attack data, the data will be limited, not rich and cause loss of balance, significantly reducing the model's performance.
- The third is the problem of analyzing and extracting contextual information: Although networks of data relationships have been created, there is still no optimization system that takes advantage of those relationships for analysis, leading to missing important information created and making it difficult to detect potential and sophisticated attacks.

To overcome the above situations, in this study we propose a new model called ACG-BT. The model is designed to improve the following limited approaches:

- First, solve the problem of synthesizing and collecting adaptive contextual information for different data: In which, AC technique is used to distill important information and self-adjust to data with different numbers of attacks, clearly demonstrating the maintenance of context and detecting unusual patterns.
- Second, generate synthetic data that simulates attack data in the network: Based on the filtered information SAG is applied on specific conditions to create new synthetic attack data to simulate hacker attack data, making the data more diverse and rich, balancing minority attack data to minimize bias and improve detection accuracy.
- Third, the problem of analyzing and extracting contextual information: BiLSTM with two-way learning will exploit the problem of context, signs surrounding attack data, sequential relationships and dependencies between data. Transformer will increase data processing capabilities by taking advantage of parallel computing and using the Multi-Head Attention mechanism to enhance the ability to focus and analyze data.

1.2 Principles of ACG-BT model operation

To detect APT attacks, the ACG-BT model will go through the following main stages:

- Phase 1: Aggregate and collect adaptive context information from network traffic. During this stage, the AC technique will determine in the entire data what information needs attention, then create frames around important data points to preserve the contextual value around the data point, and also identify information with little value

and noise information that needs to be eliminated. From there, the size of the input data is reduced but still retains the important value of the initial data to prepare for the following stages.

- Phase 2: Generate synthetic data to simulate attack data in the network. SAG with its powerful data generation mechanism will reproduce minority data to help increase the data to be detected, creating a new balanced data set, richer and more diverse attack data and relationships in the network.
- Phase 3: Analyze context and detect APT attacks. With the BiLSTM network, the model will be capable of bidirectional learning, exploiting contextual issues, sequential relationships and dependencies between data. Transformer will then increase data processing capabilities by taking advantage of parallel computing and using the Multi-Head Attention mechanism to enhance the ability to focus and analyze important data, thereby helping the model draw accurate conclusions.

1.3 Contributions of the paper

Some scientific contributions of this article include:

- Proposing a completely new ACG-BT model to improve the effectiveness of detecting APT attacks based on network traffic analysis. This is a completely new model, no previous research or articles have proposed it. The experimental results in the article have shown that this model is more effective than other models and approaches on all measures.
- Proposing a method to adapt to different data through applying AC combined with SAG. This is a new approach to the task of creating a rich and diverse dataset of adaptive contextual information from network traffic. With this proposal, the ACG-BT model succeeds in providing a deeper and clearer view of network traffic flows, highlighting potential threats masquerading as anomalous behaviors that traditional methods may miss. In addition, SAG will generate synthetic data, making data and relationships in the network more diverse, reducing bias to avoid overfitting and helping to improve the efficiency of the APT IP detection process.
- Proposing the BiLSTM model combined with Transformer to improve the ability to analyze and extract information from contextual data. BiLSTM will help the model capture relationships and dependencies in data to find sophisticated attacks hidden in common behaviors. Then apply Transformer to optimize the model's computing ability, enhancing the ability to focus and analyze data using the internal Multi-Head Attention architecture.

2 Related works

2.1 Attack detection based on analysis of events and context in the data of the aggregation model

In the paper [20], Panahnejad and Mirabi proposed APT-Dt-KC, an APT attack detection method based on the kill-chain model. This method uses the kill-chain model to analyze and identify different stages of an APT attack, thereby effectively detecting threats. Experiments show that this method achieves high accuracy, about 94%, with significant improvement compared to traditional methods. However, the research still has some challenges, including that this model requires clear identification of each attack stage, but in reality APT attacks often do not follow a fixed sequence, causing detection efficiency to decrease in real situations. Additionally, correlation-based data reduction can remove features that are less relevant to a specific target but are still valuable to other stages of an APT attack, which can lead to the loss of important information.

In the paper [21], Cho and colleagues proposed a multi-layer method to detect APT attacks based on machine learning through network traffic analysis. This research focuses on building a multi-layer analysis technique by calculating and analyzing various events in network traffic to detect and synthesize unusual signs and behaviors to draw conclusions about the existence of APTs in the system. Specifically, the study will analyze and evaluate the components of network traffic based on unusual signs and behaviors. Then build and classify behavioral profiles based on each component of traffic. Experimental results show that this method achieves an impressive accuracy of up to 92%, outperforming some traditional methods. However, the study still has some significant limitations. First, the ability to adapt to new attack variants is limited, since the model is mainly based on known features. Second, handling complex data from many different sources remains a major challenge, requiring improvements in data integration and analysis.

Research [22], Xu and colleagues developed the Proc-SAGE method, a threat detection method based on graph representation learning. This method uses deep learning to focus on processes or threads in server audit logs during the graph construction phase to control graph efficiency and reduce performance costs, and in the model feature extraction phase, it considers information about the processes themselves, threads, and their neighboring nodes to accurately characterize them to improve model accuracy. This method shows an improvement of about 5% compared to traditional methods, reaching an overall accuracy of up to 95%. But there are still some limitations that the research has not yet resolved. First, graph representation learning models are often optimized for a specific type of data, which can reduce

effectiveness when applying the method to other types of threats or environments not covered in the research. Second, the ProcSAGE method is based on representing data in graph form, which requires the input data to have clear relationships between entities. If a reasonable graph cannot be constructed or the data lacks associated information, the model's performance may suffer.

In the paper [7], Weina Niu and colleagues explored the use of deep learning combined with time series analysis and link analysis to detect malicious traffic. The research carries out improving the LSTM network into new networks such as RESNET_LSTM and PARALLEL_LSTM to extract time series features of different traffic, connecting time series feature vector and quantization vector as input to deep learning models to improve the ability to identify complex attack patterns hidden in network traffic. The accuracy of the detection methods proposed in this article are all above 96%, while the accuracy of the malware detection methods combined with static traffic characterization and machine learning is about 85%. However, some challenges still need to be addressed. Firstly, it is necessary to optimize the model to better suit new attack variants, ensuring high flexibility and adaptability. Second, time series analysis requires determining a suitable length for feature extraction. If a non-optimal length is chosen, the model may miss important information or lead to redundant data processing.

Research [23], Xiang et al. developed an APT event extraction method based on the BERT-BiGRU-CRF model. This method focuses on detecting events related to APT attacks by using a deep learning model to analyze and extract information from data. BERT is used for semantic encoding, BiGRU for sequential information processing, and CRF for event extraction optimization. The results show that the APT attack event extraction model based on BERT-BiGRU-CRF achieves high F1 value. However, the research still faces several challenges, including reliable extraction methods based on data from web documents. If the source data is incomplete, lacks accuracy, or contains erroneous information, event extraction results can be seriously affected. Furthermore, the BERT-BiGRU-CRF model is optimized for predefined APT event types, so it will be difficult to extract APT events that are not defined or do not belong to the constructed event schema.

2.2 Some other approaches

In the paper [24], Irshad et al. introduced TRACE, an enterprise-wide provenance tracking system for real-time APT detection. This system uses origin tracing techniques to record and analyze activities in the network, thereby detecting unusual behavior that may be a sign of an APT attack. TRACE is designed to operate at scale, providing fast and effective threat detection. Experimental results showed that

TRACE provided enough evidence to detect more than 80% of attack phases in all evaluations. By the last interaction, time and space costs had dropped significantly by 18% and 10%, respectively. However, the study also points out some limitations, including that TRACE uses static analysis to determine program unit structure and dependencies, but static analysis may not be effective when dealing with dynamic malware or polymorphic and self-mutating encryption. Additionally, although the number of supported system calls has increased from 47 to 66, it is still possible that there are system call types that are not covered, which may reduce the ability to detect new or previously unseen APT attack stages.

In the study [25], Neeraj Saini and colleagues proposed a hybrid ensemble machine learning model to detect APT attacks based on network behavior anomaly analysis. This model combines Random Forest and XGBoost to improve the ability to detect APT threats through analyzing network behavior. The study was tested on four popular datasets (CSE-CIC-IDS2018, CIC-IDS2017, NSL-KDD and UNSW-NB15). Experimental results show that the model achieves high accuracy, from 97 to 99%, superior to traditional methods. But the research still has some limitations. First, using stratified random sampling can lose important information, affecting the accuracy of the model. Second, although the improved false positive rate is low in most datasets, with UNSW-NB15, the false positive rate is 5.29%, which can cause significant difficulties in real network systems when having to handle many false alarms. Finally, Random Forest and XGBoost methods can have performance problems when handling very large data volumes or real-time requirements.

Chen, et al., developed APT-KGL [26], an intelligent APT detection system based on threat knowledge and heterogeneous provenance graph learning. The system uses knowledge graphs to model threats and deep learning to analyze complex relationships between entities in the graph. APT-KGL is designed to detect APT attacks through knowledge mining from different data sources. Experimental results show that APT-KGL outperforms other deep learning-based models. The research has a number of unresolved limitations. First, converting a heterogeneous graph to a homogeneous graph may lose some important information about the relationships between entities. Second, although the APT-KGL system can perform well on small datasets, scaling to larger scales with millions of data points has not been fully tested. Training and processing complex contextual data can be difficult in large-scale environments.

In research [8], Han and his colleagues studied APT attack detection technology based on the DenseNet CNN network. The first method will analyze the characteristics of the binary sample of the malicious code attack by performing several operations such as decompression and decompilation. Then use a convolutional neural network to extract features, cluster the data, and draw conclusions. Experimental results show

that the average accuracy of the proposed scheme for detecting APT malware and its variants can reach 98.84%. But the research still has some unresolved challenges. First, this method strongly depends on high-quality training data, but data related to APT attacks is often difficult to collect and fully label, potentially reducing the model's effectiveness in real-world environments. Second, DenseNet may have difficulty detecting new or never-before-seen APT attacks in training data (zero-day attacks), because deep learning models often rely on known data patterns.

In research [27], author Do Xuan Cho proposed the use of machine learning to detect APT attacks based on network traffic analysis. This method focuses on two main components: domain and ip, these two components will be used to evaluate the behavior of attacks. The behavior will then be evaluated and classified based on the Random Forest algorithm to conclude about APT attacks. Experimental results show that this method achieves good results with an accuracy of about 91%. Some limitations still exist in the study. First, anomalous behavior of Domain and IP is defined in Sect. 3.2, however if this definition is not comprehensive enough or does not cover complex variations of APT attacks, detection efficiency can be significantly reduced. Second, Random Forest-based methods can perform well on known anomalous behavior, but are less effective at detecting never-before-seen APT attacks or sophisticated attack campaigns using new obfuscation techniques.

3 ACG-BT model for APT detection

3.1 Proposed model architecture

Figure 1 represents the ACG-BT model architecture, the main components of the model include:

- Synthesize and collect contextual information adapted to the data: This block will use AC to synthesize, distill important information and adapt to data with different numbers of attacks so that the context surrounding the attack data is still maintained.
- Generating synthetic data simulating attack data in the network: In this block, SAG will use data filtered from the previous block to create data simulating attack data based on certain conditions, making the data more diverse, the relationships of data in the network become richer and rebalancing the data to minimize bias.
- Analyze and extract contextual information: using BiLSTM combined with Transformer. Blocks will help exploit the context and signals surrounding attack data, while increasing the model's ability to process and focus on important data.

- Classification: This block classifies Pair-IP based on the data that has been processed and balanced.

3.2 Aggregate and collect adaptive contextual information with AC-based data

3.2.1 Theoretical basis

AC is a time series data analysis method, inspired by the study [28] of Fawaz et al. (2019). This method focuses on identifying important points in sequence data and analyzing context by looking at previous and subsequent data points. This allows the deep learning model to recognize complex patterns and contextual relationships, thereby improving classification. The process begins by identifying important data points as the center point, then creating a context window by taking the data points before and after the center point. The relationships and interactions between these points are analyzed to identify unusual patterns. Information from points in the context window is combined using deep learning techniques, creating a framework that represents the entire context. This framework is used to detect hidden patterns and relationships, helping to clearly distinguish between normal and abnormal behaviors. The principle for creating context window frames is shown through formula (1):

$$y_t = \{\text{Central}(x_t) + \text{Contextual}(x_{t-k}, \dots, x_{t+k}) + b\} \quad (1)$$

In there:

- y_t : Data frame at time t .
- $\text{Central}(x_t)$: Important data centered at time t .
- $\text{Contextual}(x_{t-k}, \dots, x_{t+k})$: Context surrounding the data center at time t .
- b : An offset value to adjust the result.

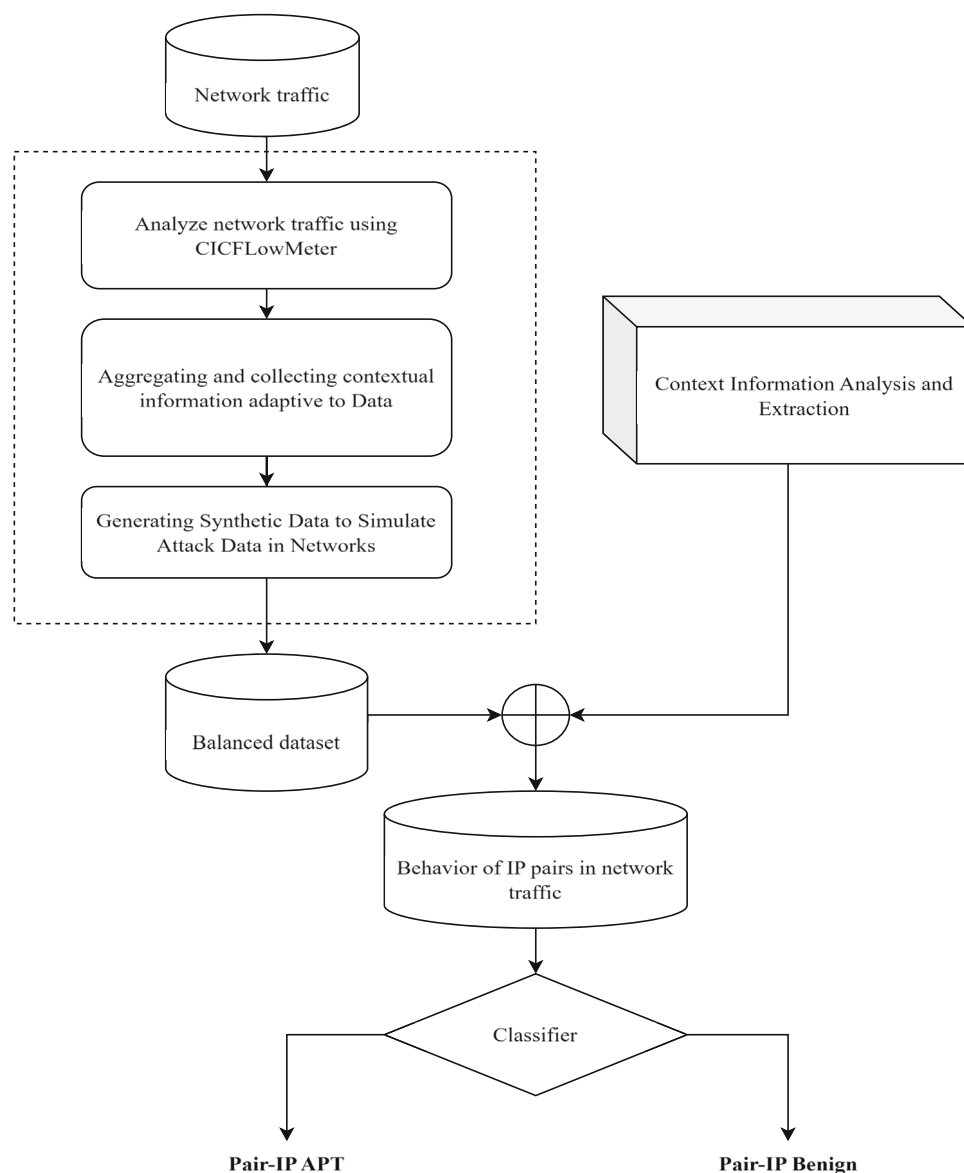
3.2.2 Aggregate and collect adaptive contextual information with AC-based data

The operating principle of the AC model used to synthesize and collect contextual information adapted to the data is described in Fig. 2.

The AC model synthesizes and collects contextual information adapted to the data as shown in Fig. 2, including the following steps:

- Step 1: Identify the data to focus on and the context: The process begins by identifying important data points (Central) and surrounding context (Contextual). Each flow will be analyzed to find meaningful flow points, these flows will form centers and then create a data frame large enough to

Fig. 1 ACG-BT model architecture for APT attack detection



surround the center and the data before and after it to create a data frame full of important data and surrounding contexts.

- **Step 2: Analyze data:** After identifying important data centers and surrounding contexts. The analysis process will take place, the goal of this step is to analyze data that is far from the center and not in the context area. Make sure to clearly identify which data is likely to be a confounding factor that needs to be eliminated, and which data, although far away, is still valuable and needs to be added to the frames to serve the analysis process in the following steps.
- **Step 3: Eliminate redundant data:** Once you have analyzed and identified the part that needs to be removed. This step

will remove noisy data or unnecessary contextual information. This process will help optimize resources and focus on the information that the problem needs to care about, while reducing data complexity.

- **Step 4:** Finally, the remaining data will be aggregated into a complete data frame. This data is smaller in size than the original data but still ensures full detailed information of the components, even eliminating disturbing factors to help the model learn the data to focus on and the context around it, thereby analyzing the surrounding impact relationships and interdependencies of the data to create a more comprehensive view for the model later.

Based on the four steps presented, we find that this approach brings many benefits to the system. The process of

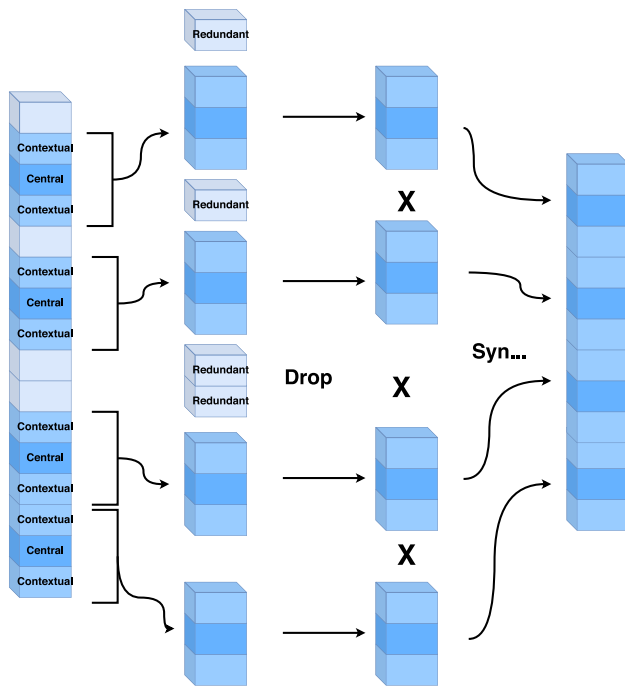


Fig. 2 Process of synthesizing and collecting context information adapted to AC data

identifying and focusing on important data points along with the surrounding context helps the model have a more comprehensive and in-depth view of the data. Removing redundant and noisy data not only optimizes resources but also improves model accuracy and performance. By synthesizing data into a complete framework, the model can easily identify and analyze complex relationships and interdependencies between data elements. This creates a solid foundation for detecting and predicting unusual patterns or potential threats, thereby improving system reliability and efficiency in handling complex situations.

3.3 Synthetic data generation simulates attack data in SAG-based networks

3.3.1 Theoretical basis

SAG in research to generate network attack simulation data is built based on CGANm [29]. This is a variation of Generative Adversarial Networks (GAN), designed to improve control over the output of generative models. The structure of SAG includes the following main components:

- **Generator(G):** The input to G is a random vector z typically sampled from a Gaussian distribution and a condition vector c containing additional information such as labels. G's task is to generate data. To do this, G uses a neural network to transform z and c into new data ($G(z|c)$) with the goal

of creating data that, when combined with conditions, will closely resemble real data.

- **Discriminator(D):** The input of D receives real data x and generated data ($G(z|c)$) along with condition information. D will be a neural network whose task is to learn to distinguish between real data and fake data. Prediction probability ($D(x|c)$) shows that the data is real under the conditions.

The loss function is described based on formula (2):

$$\min_G \max_D V(D, G) = E_{x \sim p_{data(x)}} [\lg D(x|c)] + E_{z \sim p_{z(z)}} [\lg (1 - D(G(z|c)))] \quad (2)$$

In there:

- $E_x \sim p_{data(x)} [\lg D(x|c)]$: Expectation of the log probability that the discriminator correctly predicts the real data x under condition c .
- $E_z \sim p_{z(z)} [\lg (1 - D(G(z|c)))]$: Expectation of the log probability that the discriminator incorrectly predicts the spurious data $G(z|c)$ under condition c .

The training process is like G and D participating in a two-sided game. The goal of D is to maximize the ability to distinguish real data from fake data. G's goal is to minimize D's ability to distinguish fake data, generating data that makes D unable to distinguish real data from fake data and can only guess randomly with probability 0.5.

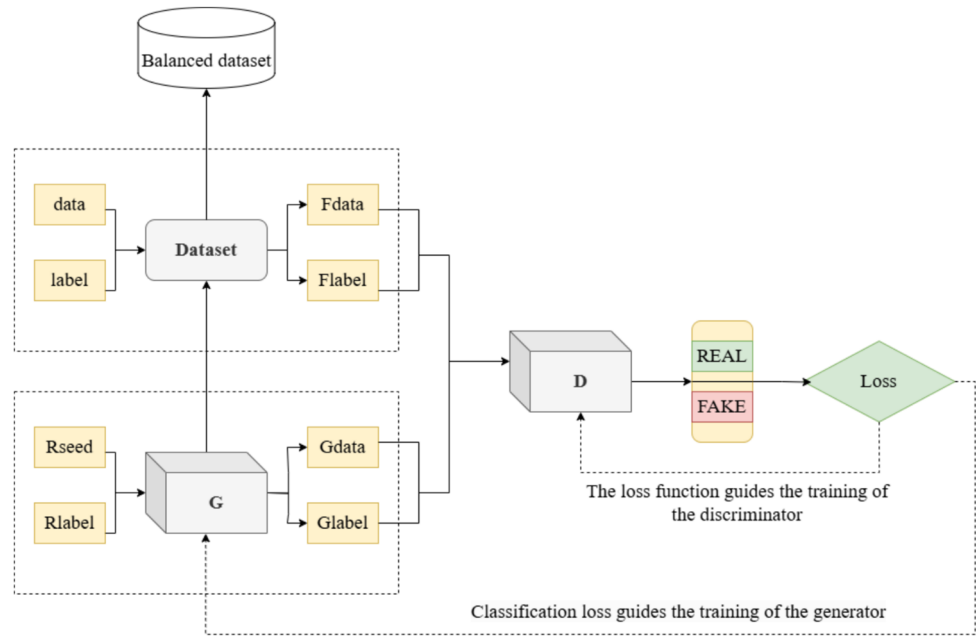
3.3.2 Generate synthetic data to simulate attack data in SAG-based networks

The operating principle of the SAG model used to generate synthetic data is described in Fig. 3:

The SAG model generates synthetic data to simulate attack data in the network as described in Fig. 3, including the following steps:

- **Step 1: Prepare condition data and generate synthetic data:** Prepare random vector R_{seed} and condition vector R_{label} . The R_{label} is processed through the Flatten and Embedding classes to create the appropriate format. Then use the multiply class to combine R_{seed} and R_{label} to form input for G. G will learn the distribution rules of data with attack behavior and data with normal behavior from which G will create attack samples that extend G_{data} and the corresponding label G_{label} that are similar to the real distribution data.
- **Step 2: Prepare real data and differentiate data:** The real data F_{data} and the corresponding label F_{label} are normalized and transformed in accordance with the input of

Fig. 3 Synthetic data generation process simulating attack data in SAG's network



D. This data often has an uneven distribution in the network. D will receive input as Gdata and Glabel, Fdata and Flabel to distinguish between real data and generated data, trying to minimize the loss function.

- Step 3: Training and optimization: The results of the loss function during training D are used to optimize G and D. If the loss of D is high, it shows that D is having difficulty distinguishing between real data and generated data. Conversely, D's low loss means that G needs to improve to deceive D better. G will try to improve to deceive D, D will try to improve its ability to discriminate. The training process continues until a "Nash" equilibrium is reached, where both models perform optimally.
- Step 4: Create a balanced dataset: Once G has been optimized the Gdata data samples are integrated with the real Fdata data to create a balanced data set. This balanced dataset ensures diversity and richness of data, creating new relational networks that improve model performance in detection and classification.

This process helps build a system that is robust and efficient in handling imbalanced data. By using random vectors and condition vectors to generate synthetic data, the system creates rich and diverse data samples, accurately reflecting the characteristics of real data. Integrating the generated data and real data to create a dataset containing more relationship networks is very important in detecting sophisticated attack patterns or hidden in normal data, along with balanced data helps minimize the problem of data imbalance, thereby improving the performance of the model in detection and classification.

3.4 Analyze and extract contextual information based on BiLSTM combined with Transformer

3.4.1 Theoretical basis

3.4.1.1 Introducing BiLSTM BiLSTM [30] is an improved neural network architecture of Long Short-Term Memory (LSTM), which allows data processing in both directions: front-to-back and back-to-front. This structure helps BiLSTM capture long-term and short-term contextual relationships in data, improving its ability to identify unusual patterns compared to traditional neural networks. BiLSTM is built with a forward LSTM and a reverse LSTM, allowing to learn features from data in both directions. Each LSTM unit consists of three main gates: input gate, forget gate, and output gate. These ports help regulate the information transmitted across the network, ensuring important features are not lost during the learning process.

In the proposed model, BiLSTM is used to extract features from network traffic data, helping the model to detect unusual behaviors related to APT attacks, providing a more comprehensive view of the data chain, enhancing the ability to detect sophisticated and complex attack patterns.

Formula (3) describes the process of calculating the output of a BiLSTM layer as follows:

$$\begin{aligned} h_t^{\text{forward}} &= \text{LSTM}_{\text{forward}}(x_t, h_{t-1}^{\text{forward}}) h_t^{\text{backward}} \\ &= \text{LSTM}_{\text{backward}}(x_t, h_{t+1}^{\text{backward}}) \end{aligned} \quad (3)$$

In there:

- $h_t^{forward}$ and $h_t^{backward}$ is the output of LSTM in the forward and reverse direction at time t .
- x_t is the input at time t .
- $LSTM_{forward}$ and $LSTM_{backward}$ are LSTM networks running in the forward and reverse directions

3.4.1.2 Introducing transformer Transformer is a neural network architecture that was first introduced in the paper [31] "Attention is All You Need" by Vaswani et al. in 2017. Transformer revolutionized the field of natural language processing (NLP) and was later widely applied in many other fields thanks to its ability to process sequential data efficiently and powerfully. Unlike traditional sequential models such as RNN or LSTM, Transformer does not rely on sequential structures to process data. Instead, it uses the Attention mechanism to determine the importance of each element in the data sequence, allowing the model to focus on the most important parts without having to process each step sequentially. This gives the Transformer parallel processing capabilities, increasing training speed and improving performance. The Attention mechanism is a core component of Transformer, allowing the model to learn to focus on the most relevant parts of the data. There are three main components in the Attention mechanism: Query (Q), Key (K), and Value (V). Attention is calculated by comparing the Query with the Key to create a weight, then applying this weight to the Value to produce the output. Formula (4) to calculate Attention can be expressed as follows:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q * K^T}{\sqrt{d_k}}\right) * V \quad (4)$$

In there:

- Q : Query matrix.
- K : Key matrix.
- V : Value matrix.
- d_k : Size of the key vector.
- $Q * K^T$: Scalar product of query matrix and key.

3.4.2 Analyze and extract contextual information based on BiLSTM combined with transformer

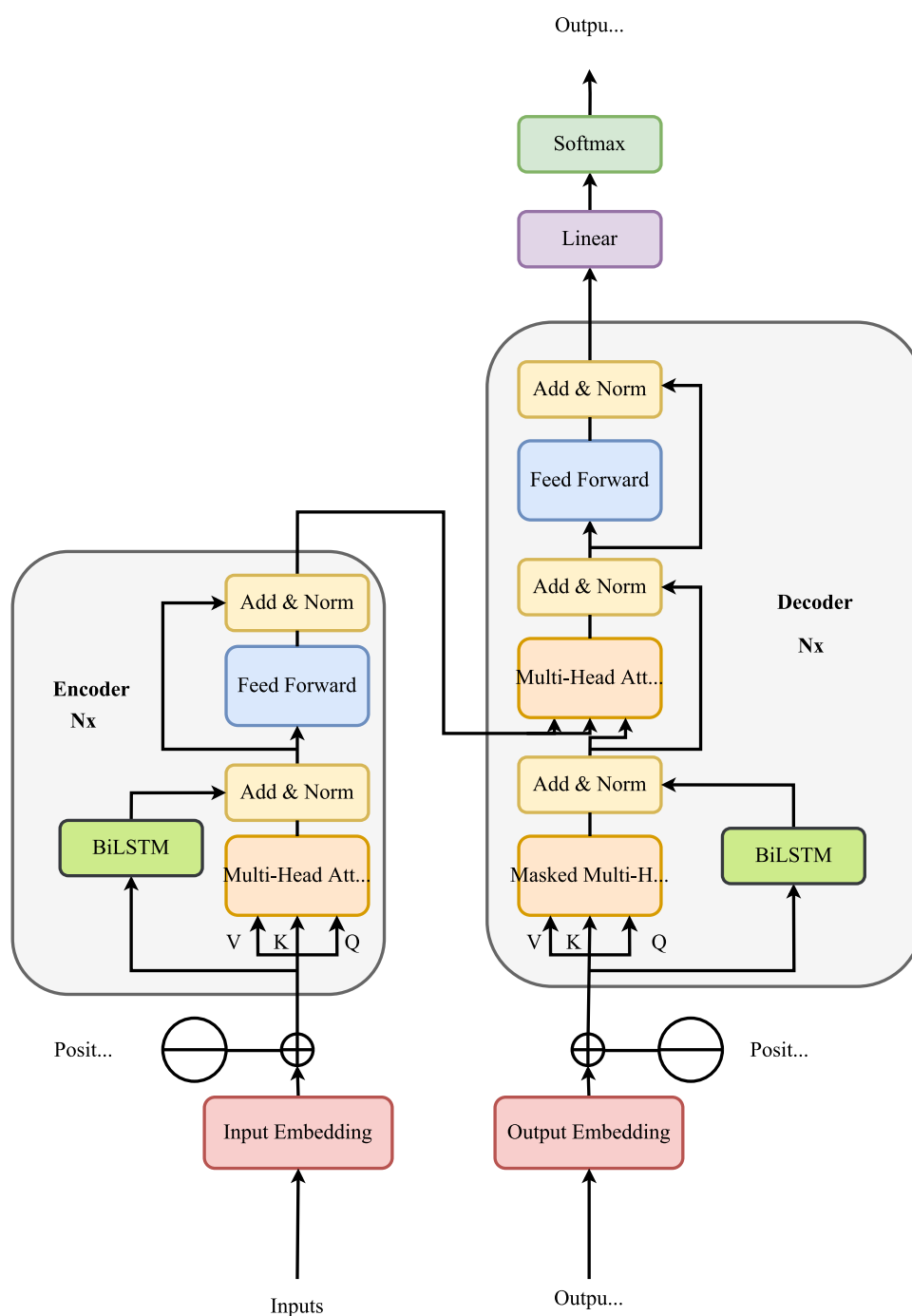
The operating principle of the BiLSTM model combined with Transformer used to analyze and extract contextual information is described in Fig. 4:

The BiLSTM-Transformer model performs analysis and extraction of contextual information as described in Fig. 4 including the following steps:

- Step 1: Encode input and process via BiLSTM in Encoder: The input data is prepared by normalizing and adding Positional Encoding to maintain the sequence information of the sequence. The prepared data will be passed through the BiLSTM layer. BiLSTM processes data in both directions, from front to back and vice versa, to detect sequential relationships in context and constraints in data.
- Step 2: Apply Multi-Head Attention and Feed Forward in Encoder: After going through BiLSTM, the data is processed through the Multi-Head Attention mechanism. Attention helps the model focus on important parts of the data, identifying necessary relationships for analysis. The output from Attention is normalized and combined with the original input via Add&Norm. The data will then be passed through Feed Forward, a feed-forward neural network, which enhances the model's analytical capabilities by processing additional features from the data. The result is then further normalized and combined with the original input via Add&Norm.
- Step 3: Process through BiLSTM and apply Masked Multi-Head Attention in Decoder: The output from the Encoder is fed into the decoder and processed through the BiLSTM layer. Masked Multi-Head Attention is then applied to ensure that at each step the model only considers previous information, preventing seeing future information because it is very important in the sequential prediction task. The result is then normalized and combined with the original input via Add&Norm.
- Step 4: Apply Multi-Head Attention from encoder in decoder and process via Feed Forward in decoder: Decoder further uses Multi-Head Attention which allows the model to use encoded information to improve predictions, the results are then normalized and combined with the original input via Add&Norm. The data is then also fed through Feed Forward, which enhances the ability to analyze and prepare data for the final prediction step, and the normalized results continue to be combined with the original data through Add&Norm. The final data will be standardized to the classification step.

Combining the BiLSTM-Transformer model in analyzing and extracting contextual information is a reasonable and effective choice. Because BiLSTM helps capture sequential relationships and long-term dependencies in data by processing in both directions, this is important for understanding context and complex relationships. Multi-Head Attention allows the model to focus on important parts of data, identifying relationships necessary for analysis, thereby improving accuracy and efficiency in detecting sophisticated attack patterns. The combination of Attention and Feed Forward in both Encoder and Decoder enhances data analysis and processing capabilities, ensuring that the model can handle complex features and prepare data for the final

Fig. 4 Operating principle of BiLSTM-Transformer



prediction step. Masked Multi-Head Attention in Decoder ensures that the model only considers previous information, which is important in sequential prediction tasks, helping to improve prediction accuracy. Thanks to this combination, the BiLSTM-Transformer model not only improves analysis and prediction capabilities but also ensures comprehensiveness and accuracy in processing complex data. This makes the model a powerful tool in detecting and classifying potential threats.

3.5 Pair-IP classification

For Pair-IP classification, we use two main layers: Fully Connected Layer and Softmax Layer:

- **Fully Connected Layers:** Receive features processed through the BiLSTM-Transformer block then learn these features and convert them into suitable representations for classification

- **Softmax Layers:** The Softmax layer takes on the role of calculating the probability for each APT or Benign class, helping to determine the likelihood of belonging to each class of a specific Pair-IP. Softmax calculates probabilities by converting the output values into probabilities in the interval $[0, 1]$, ensuring the sum of the probabilities is 1. Formula (5) shows how to calculate the probability for class i :

$$P_i = \frac{\exp(z_i)}{\sum_{j=1}^C \exp(z_j)} \quad (5)$$

In there:

- P_i is the probability that the input belongs to class i .
- z_i is the output from the Fully Connected class corresponding to class i .
- C is the total number of layers.

The result of the Softmax layer will determine the label of each data sample by selecting the class with the highest probability. This helps classify Pair-IPs as at risk of APT attacks or normal, ensuring the model can accurately detect potential threats.

4 Experiment and evaluate

4.1 Experimental data

4.1.1 Data collection

The data set includes clean and malicious network traffic, and the malicious part was collected and analyzed from 29 files in the Malware Capture-13 set of 6 types of malware in APT attacks including: Andromeda, Colbalt, Cridex, Dridex, Emotet and Gh0stRAT [32]. The clean network traffic part is extracted from the e-government server of Quang Nam province [33] according to the scientific research project KC.01.05/16-20 of the Ministry of Science and Technology of Vietnam. This dataset was collected on one day, July 27, 2019.

4.1.2 Data preprocessing

The initial data set will be passed through CICFlowMeter to extract flow network data. The raw flow data will now be preprocessed to be included in the model. Flows containing nan or infinity values will be dropped, and duplicate flows will be removed, leaving only the single flow. Fields that do not provide value when

training such as: 'publicIP', 'FlowID', 'SrcPort', 'DstPort', 'Timestamp', 'Protocol', 'FlowDuration' will be omitted. Table 1 below is details about the data set used in the article's experiments.

4.2 Experimental scenario

In this article, we will use the ACG-BT model to answer some of the following research questions (RQ):

- **RQ1:** Why does the ACG-BT model perform well in the task of aggregating and collecting adaptive contextual information on data? To answer this question, the article will conduct experiments to compare and evaluate the proposed model with a number of other models and approaches in the task of synthesizing and collecting information.
- **RQ2:** Why does the ACG-BT model perform well in the task of handling imbalanced data? In this scenario, we will compare this research with some other models in the same approach such as: VAE-LSTM [34], KNN-SMOTE-LSTM [35].
- **RQ3:** Why does the ACG-BT model perform well in the task of analyzing and extracting contextual information in data?

4.3 Evaluation criteria

Accuracy:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \times 100\% \quad (6)$$

Precision:

$$\text{Precision} = \frac{TP}{TP + FP} \times 100\% \quad (7)$$

Recall:

$$\text{Recall} = \frac{TP}{TP + FN} \times 100\% \quad (8)$$

F1-score:

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (9)$$

where:

TP—True positive: The number of malicious samples classified correctly.

FN—False negative: The number of malicious samples classified as normal.

TN—True negative: The number of normal samples classified correctly.

Table 1 Experimental data set composition

Full	Type	Total	APT	BENIGN
1	Flows	1,671,393	871,914 (52.17%)	799,479 (47.83%)
2	Pair-IP	1,884	218 (11.57%)	1,666 (88.43%)

Table 2 Results of detecting APT attacks when synthesizing and collecting contextual information adapted to data?

Model				Evaluation of IP			
				Acc	Pre	Rec	F1
ACG-BT				0.9945	0.9811	0.9858	0.9835
Window (number of flows before and after APT center)	SAG latten dims	BiLSTM layers	Transformers layers				
10	100	2	3				
CNN-LSTM [36]				0.95	0.94	0.90	0.92
Number of CNN layers		Number of LSTM layers					
3		2					
MIG [17]				0.9732	0.9731	0.9742	0.9740
Number of MLP layers	Inference β coefficient		Number of GCN layers				
3	0.7		2				

FP—False positive: The number of normal samples classified as malicious.

4.4 Experimental results

4.4.1 Experimental results of scenario 1

Table 2 below shows the experimental results of scenario 1. Note that the results shown in this table are the best results extracted from the experimental process.

From the results in Table 2 it can be seen that the ACG-BT model with accuracy 99.45%, Precision 98.11%, Recall 98.58%, and F1-score 98.35% or more outperforms other models such as CNN-LSTM and MIG. The explanation for achieving such results is due to a number of reasons. First, AC eliminates noisy and mostly worthless information to help optimize resources, and the model can focus on learning important information and the context around it to detect sophisticated attack patterns more effectively. Second, synthesizing data into a complete framework is like creating a comprehensive picture, helping the model easily analyze the relationships and interactions between data components.

Compared with CNN-LSTM, CNN-LSTM, although popular, only achieves 95% accuracy, 94% Precision, 90% Recall, and 92% F1-score. The main reason why CNN-LSTM does not achieve as high performance as ACG-BT. First, limited to long-term context processing, CNN-LSTM combines the spatial feature extraction capabilities of CNN and the sequential processing of LSTM but cannot fully capture the long-term and complex relationships in the data.

This leads to missing sophisticated attack patterns. Second, lacking the ability to focus flexibly, CNN-LSTM lacks a mechanism to eliminate redundant data and retain important data and context, causing noise that reduces the ability to focus on important parts of data.

Compared with MIG, MIG achieves accuracy 97.32%, Precision 97.31%, Recall 97.42%, and F1-score 97.40%. The limitations of MIG include first, depending on the inference layer, MIG uses the Inference layer to synthesize many flows into one and gives high weight to APT flows to focus on important data. However, this approach can lead to missing the complex relationships between flows because it only focuses on weights without considering the broader context. Second, because it does not eliminate noisy data but only focuses on aggregation ability, when a large amount of noisy data is aggregated, it will obscure the important information value, which affects the performance of the model.

The confusion matrix in Fig. 5 illustrates the impressive performance of the ACG-BT model in classifying labels. Specifically, the model correctly identified 208 attack cases (True Positives) and 1055 normal cases (True Negatives). This demonstrates the model's ability to correctly identify attack and non-attack patterns.

Notably, only 4 normal cases were mistakenly predicted as attacks (False Positives) and 3 attack cases were missed (False Negatives). This low number of false predictions reflects a good balance between sensitivity and specificity of the model. Maintaining a low False Positives and False Negatives ratio is critical in the context of detecting APT threats, where minimizing false alarms and misses can help improve

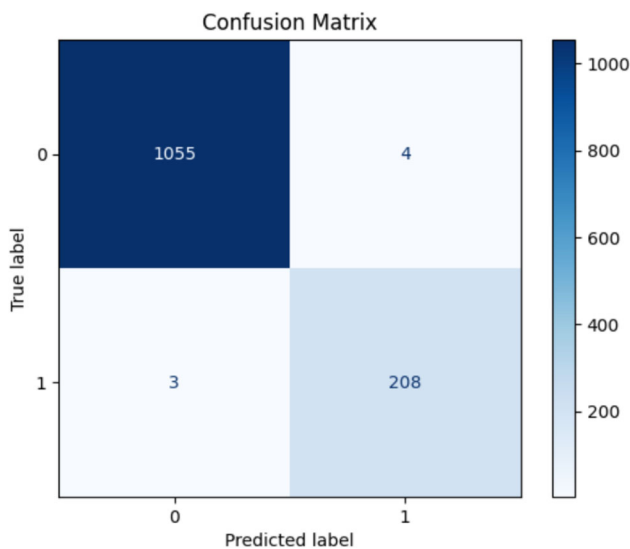


Fig. 5 Confusion matrix of the ACG-BT model with the best results

overall system reliability. This result confirms the effectiveness of the ACG-BT model in detecting threats accurately and effectively. Some other extended results to evaluate the model are added in the appendix.

4.4.2 Experimental results of scenario 2

The results in Table 3 show that the ACG-BT model achieves outstanding performance in detecting APT attacks when handling imbalanced data thanks to the use of SAG to generate synthetic data. With accuracy 99.45%, Precision 98.11%, Recall 98.58%, and F1-score 98.35%, ACG-BT shows superiority compared to other models such as VAE-LSTM and KNN-SMOTE-LSTM. The first reason is to generate diverse synthetic data. SAG starts by preparing random vectors and condition vectors to generate synthetic data. This process creates rich and diverse data samples that accurately reflect the characteristics of real data, helping the model detect sophisticated attack patterns or hidden in normal data. Second, distinguish and optimize data, SAG uses real data and generated data to optimize the discriminant model, ensuring that the generated data is nearly identical to real data. Continuous training and optimization help improve the model's ability to discriminate and generate data. Third, creating a balanced dataset, SAG integrates generated data and real data to create a balanced dataset. This not only mitigates the problem of data imbalance but also creates more relational networks, improving the model's performance in detection and classification.

Compared with VAE-LSTM, VAE-LSTM achieves 97% accuracy, 84% Precision, 87% Recall, and 85% F1-score. Although VAE-LSTM uses Variational Autoencoder to generate synthetic data, it has some limitations. First, the ability

to generate data is limited, VAE may not generate rich and diverse data. This results in the model not being able to fully reflect the complex characteristics of real data, reducing the ability to detect sophisticated attack patterns. Second, limited in its ability to analyze context, VAE-LSTM may lack the ability to analyze deep context, which SAG in ACG-BT does better thanks to the optimization and differentiation of generated and real data.

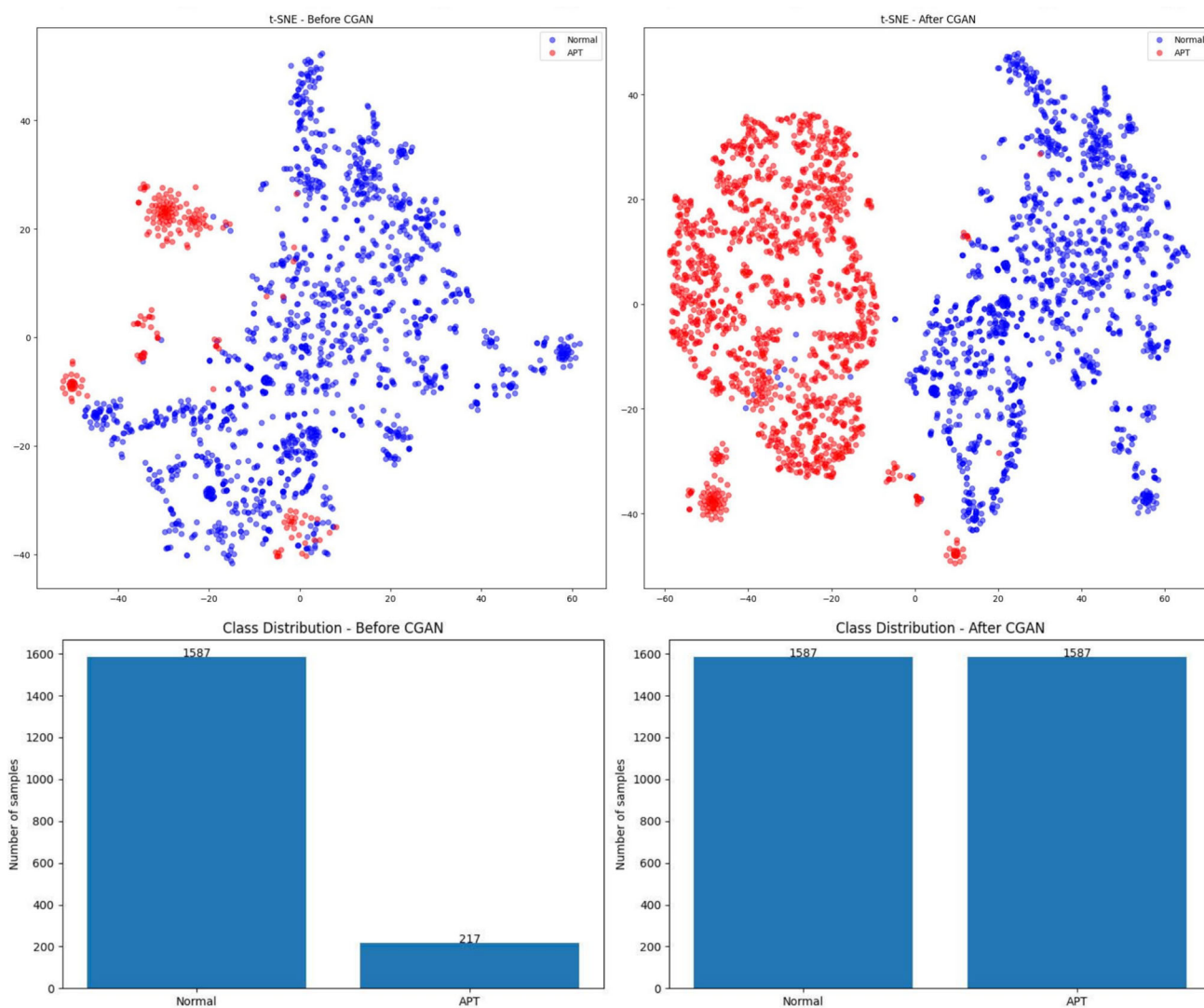
Compared with KNN-SMOTE-LSTM, KNN-SMOTE-LSTM achieves 96% accuracy, 72% Precision, 93% Recall, and 81% F1-score. Some limitations of the model when implemented. First, the simple data balancing method, SMOTE generates synthetic data by interpolating between existing data points, which may fail to capture complex and multidimensional relationships leading to poorer attack detection capabilities. Second, lacking the ability to continuously optimize, KNN-SMOTE-LSTM does not have a continuous optimization mechanism like SAG, reducing the ability to tune and improve the model during training.

Figure 6 illustrates the effectiveness of the ACG-BT model in handling imbalanced data through applying SAG. Before using SAG, the t-SNE plot showed an uneven distribution between classes, with very few APT samples (red) compared to normal samples (blue). This is also evident in the class distribution histogram, where the number of APT samples is significantly smaller. After applying SAG, the t-SNE plot shows a more even distribution between APT and normal samples, with more APT data points generated, helping to balance the data. The class distribution chart after SAG also shows that the number of APT and normal samples become equivalent, with each class having 1567 samples. These changes demonstrate that the ACG-BT model, through the use of SAG, is successful in handling the data imbalance problem. This not only improves model accuracy but also enhances the ability to detect APT threats, by providing a more diverse and representative dataset.

Figure 7 shows the change in correlation between the properties of the data before and after applying SAG. Before using SAG, attributes were unevenly correlated, with some pairs of attributes showing very strong relationships, while other pairs had almost no correlation. After applying SAG, the correlogram shows a more uniform distribution among the attributes. This shows that SAG not only generates synthetic data to balance the number of samples, but also helps adjust the relationships between attributes, making the data more diverse and representative. Improvements in attribute correlation can help the ACG-BT model capture more complex relationships in data, thereby improving its ability to detect and classify APT threats. This emphasizes the important role of SAG in optimizing input data for the model, ensuring that the data is not only balanced in quantity but also in information quality. Some other extended results to evaluate the model are added in the appendix.

Table 3 Results of detecting APT attacks when handling imbalanced data

Model				Evaluation of IP			
				Acc	Pre	Rec	F1
ACG-BT				0.9945	0.9811	0.9858	0.9835
Window (Number of flows before and after APT center)	SAG Latten dims	BiLSTM Layers	Transformers Layers				
10	100	2	3				
VAE-LSTM [34]				0.97	0.84	0.87	0.85
Number of VAE layers		Number of LSTM layers					
2		3					
KNN-SMOTE-LSTM [35]				0.96	0.72	0.93	0.81
Number of MLP layers	SMOTE		Number of GCN layers				
2			3				

**Fig. 6** Data distribution chart before and after applying SAG

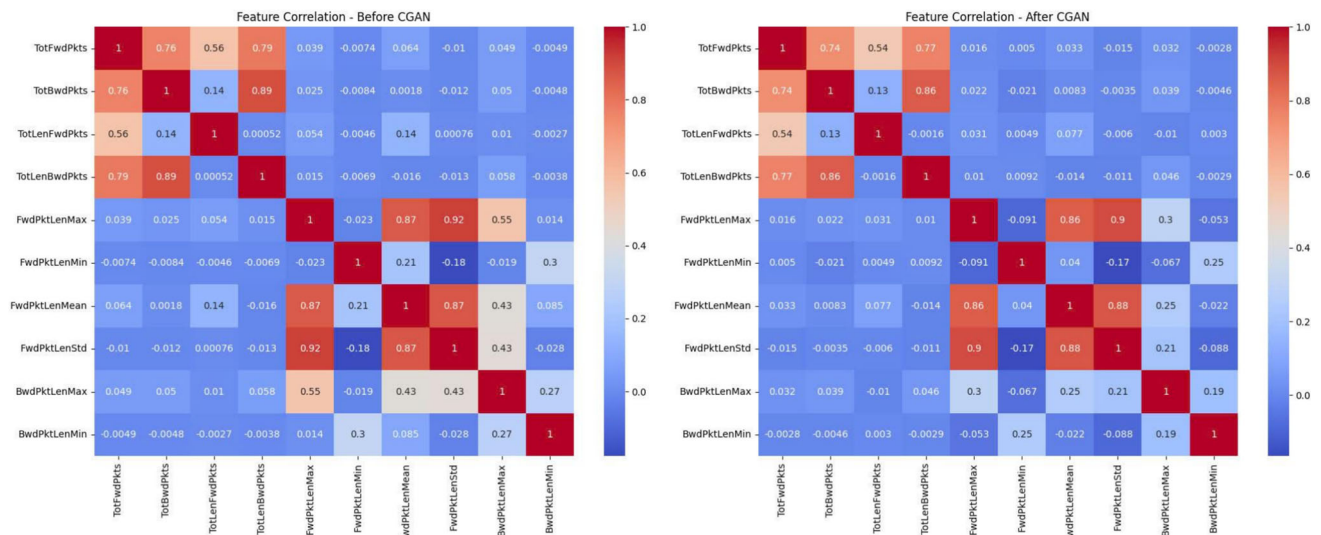


Fig. 7 Correlation chart of data properties before and after applying SAG

4.4.3 Experimental results of scenario 3

Table 4 shows that the ACG-BT Model achieves outstanding performance in detecting APT attacks thanks to the combination of BiLSTM-Transformer. With accuracy 99.45%, Precision 98.11%, Recall 98.58%, and F1-score 98.35%, ACG-BT outperforms other models such as BiLSTM-Attention and GCN-BiLSTM. Achieving such performance has several reasons. First, encoding and processing context, BiLSTM processes data in both directions, helping to capture sequential relationships and long-term dependencies in the data. Second, focusing on important information, Multi-Head Attention allows the model to focus on important parts of data, identifying relationships necessary for analysis. Third, comprehensive processing and analysis, the combination of Attention and Feed Forward in both Encoder and Decoder helps enhance data analysis and processing capabilities,

ensuring that the model can handle complex features and prepare data for the final prediction step.

Compared with BiLSTM-Attention, BiLSTM-Attention achieves 97% accuracy, 86% Precision, 86% Recall, and 86% F1-score. Although using Attention to focus on important pieces of data, this model has some limitations. First, limited in long-term context processing, BiLSTM-Attention lacks the ability to process long-term and complex contexts like BiLSTM-Transformer. This leads to missing complex, long-term relationships in the data. Second, the analysis ability is not optimal, this model does not have a strong Feed Forward mechanism like in Transformer, reducing the ability to analyze and process complex features.

Compared with GCN-BiLSTM, GCN-BiLSTM achieves accuracy 97%, Precision 89%, Recall 90%, and F1-score 89%. The model achieves such performance due to some limitations. First, lacking sequential processing capabilities,

Table 4 Results of detecting APT attacks when analyzing and extracting contextual information in data

Model	Evaluation of IP			
	Acc	Pre	Rec	F1
ACG-BT	0.9945	0.9811	0.9858	0.9835
Window (Number of flows before and after APT center)				
10				
SAG Latten dims				
100				
BiLSTM Layers				
2				
Transformers Layers				
3				
BiLSTM-Attention [37]	0.97	0.86	0.86	0.86
Number of BiLSTM layers				
2				
GCN-BiLSTM [38]	0.97	0.89	0.90	0.89
Number of GCN layers				
3				
Number of BiLSTM layers				
2				

GCN-BiLSTM cannot fully capture sequential and contextual relationships like BiLSTM-Transformer. This reduces the ability to detect sophisticated attack patterns. Second, the ability to focus is limited, this model lacks a Multi-Head Attention mechanism to focus on important parts of data, leading to lower performance in detecting threats. Some other extended results to evaluate the model are added in the appendix.

4.5 Discussion

Based on different evaluation scenarios, we have shown the superiority of the ACG-BT model compared to other methods, this superiority comes from three main reasons:

- First, adaptive context analysis. AC allows the model to adapt to different network traffic patterns. At the same time, it helps filter out noise and focus on important information, thereby improving the ability to detect subtle abnormalities. Maintaining context is key to understanding complex relationships in data, helping the model more accurately identify signs of APT attacks.
- Second, balance the data with SAG. SAG plays an important role in generating aggregated data, helping to balance datasets and overcome the imbalance problem. By simulating new attack patterns, SAG provides a rich and diverse data set, helping the model learn more from different attack scenarios. This not only improves accuracy but also enhances the ability to detect previously unseen attack patterns.
- Third, contextual and sequential analysis, BiLSTM and Transformer are combined to exploit both short-term and long-term dependencies in the data. BiLSTM processes data in both directions, front to back and vice versa, helping to capture context and sequential relationships. Meanwhile, Transformer with Multi-Head Attention mechanism enhances the ability to focus and analyze data, allowing the model to process large amounts of information effectively.

Experimental results show that ACG-BT outperforms existing models by 2% to 4% on all evaluation metrics. This shows that the combination of adaptive and adversarial techniques has brought remarkable effectiveness in detecting APT. The model not only improves accuracy but also ensures comprehensiveness in handling complex data. However, the model still has some limitations:

In some special situations, such as when faced with extremely complex or continuously changing structured network data, AC may have difficulty capturing all the hidden relationships between data points. Imagine a large enterprise network, where tens of thousands of devices from computers, servers, to IoT devices all operate at the same time, using many different protocols such as HTTP, FTP,

or custom protocols. Data from these environments is often heterogeneous—sometimes contaminated by factors such as transmission errors, hardware failures, or even legitimate but unusual user activities. As a result, the performance of the model may be slightly affected, especially in the early stages when new data is introduced. AC's dependence on the quality and consistency of input data is the main reason. When data is noisy, fragmented, or does not follow a clear pattern, it becomes more difficult to build accurate context. This is a natural challenge in real network environments, which are rarely stable or predictable.

Regarding SAG issues, SAG effectively enhances data richness and supports the cyber threat detection model. However, we recognize that SAG sometimes needs a little more time to adapt to new attacks, especially threats that have never appeared before, such as zero-day attacks with completely different characteristics. Imagine a cyberattack that uses a novel method, such as a unique way of encrypting data or exploiting a vulnerability not yet recognized by the security community. In such cases, SAG—which is trained on known data patterns—may need more time to adapt and generate synthetic data that matches these new features. This does not mean that SAG is not doing well, but just that it is in the process of "learning" to keep up with the constant innovation of cyber threats. This limitation stems from SAG's reliance on historical data samples to construct aggregate data. When faced with completely novel threats, SAG does not yet have enough data to instantly create accurate synthetic data. Additionally, fine-tuning SAG parameters to produce more diverse data sometimes requires careful adjustment, as a balance needs to be struck between maintaining accuracy and exploring new scenarios.

5 Conclusion

In this study, in order to improve the ability to accurately detect APT threats, we have proposed and successfully evaluated the ACG-BT model, based on the combination of advanced techniques. Experimental results demonstrate the effectiveness of the ACG-BT model in three important tasks: analyzing and extracting context from network traffic, balancing attack data, and detecting sophisticated attack patterns in complex datasets. Based on different evaluation experimental scenarios, we have provided an overview and scientific basis to build an effective APT detection system, using advanced deep learning and machine learning techniques. In the future, based on the experimental results of this study, we believe that some contents need to be improved: Vissue issue Improve AC disadvantages: To overcome, we are considering integrating some advanced techniques. The first is to use intelligent noise filters, such as deep learning-based algorithms to remove irrelevant data before feeding

it into AC. Second, unsupervised learning methods can be applied to help AC automatically identify unusual data patterns without relying entirely on labeled data. In addition, optimizing algorithms to process large volumes of data in real time is also a promising direction. These improvements not only help AC operate more stably but also turn it into a more powerful tool, ready to face all challenges in complex network systems. Regarding improving SAG's shortcomings, mOne promising direction is the integration of transfer learning, which allows SAG to learn from data patterns or related systems to create richer synthetic data. For example, SAG can refer to attack patterns from similar domains to predict novel features. In addition, applying unsupervised learning or reinforcement learning is also worth considering, as they can help SAG explore and create simulation scenarios on its own without completely depending on old data. These moves will not only help SAG keep pace with new threats, but will also transform it into a more innovative tool, ready to help ACG-BT meet any future challenges. These improvements will make ACG-BT a more powerful tool in protecting cybersecurity against increasingly sophisticated APT attacks.

Appendix

See appendix Figs. 8, 9, 10 and Tables 5, 6, 7, 8, 9, 10, 11, 12, 13, 14 and 15.

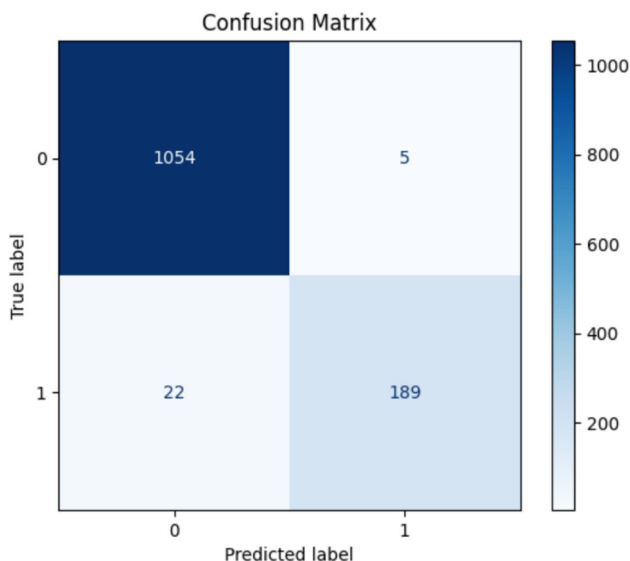


Fig. 8 Confusion matrix evaluating ACG-BT when replacing AC with Inference

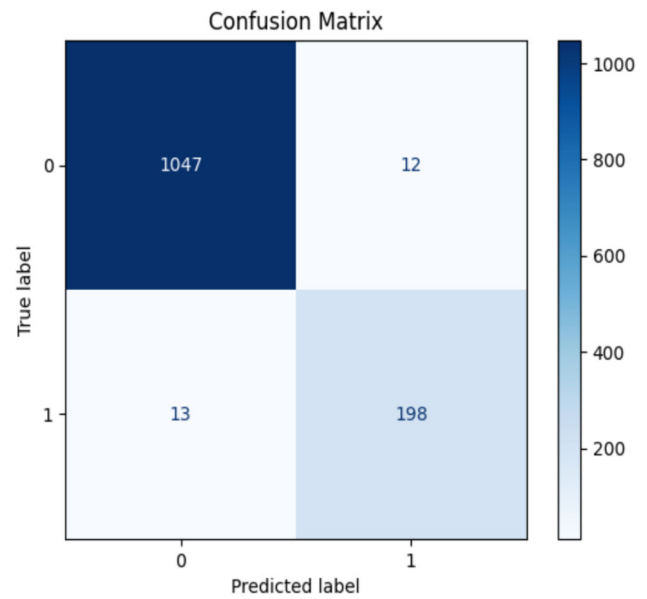


Fig. 9 Confusion matrix evaluating ACG-BT when replacing SAG with VAE

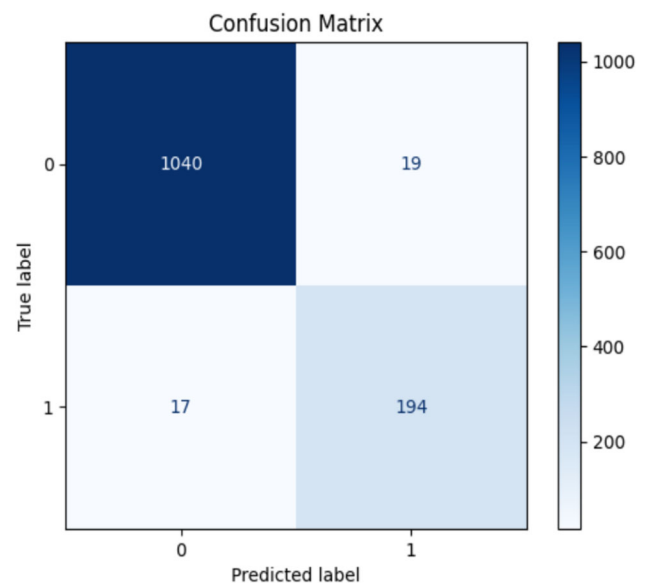


Fig. 10 Confusion matrix evaluating ACG-BT when replacing BiLSTM + Transformer with BiLSTM + Attention

Table 5 APT attack detection results of ACG-BT when replacing AC with grouping by Frame

Flows of Frame	SAG Latten dims	CNN Layers	LSTM Layers	Evaluation of IP			
				Acc	Pre	Rec	F1
30	100	2	3	0.8157	0.7418	0.9685	0.8401
50	100	2	3	0.8469	0.7933	0.9382	0.8597
100	100	2	3	0.8447	0.7799	0.9603	0.8608

Table 6 APT attack detection results of ACG-BT when replacing AC with Inference

Inference	SAG Latten dims	BiLSTM Layers	Transformers Layers	Evaluation of IP			
				Acc	Pre	Rec	F1
0.3	100	2	3	0.8106	0.7584	0.9118	0.8280
0.5	100	2	3	0.8655	0.8201	0.9364	0.8744
0.7	100	2	3	0.9787	0.9742	0.8957	0.9333

Table 7 APT attack detection results for ACG-BT without SAG

Window (Number of flows before and after APT center)	BiLSTM Layers	Transformers Layers	Evaluation of IP			
			Acc	Pre	Rec	F1
10	1	2	0.9285	0.7245	0.6544	0.6877
10	2	3	0.9501	0.8191	0.7512	0.7837
10	3	4	0.9302	0.7156	0.6959	0.7056

Table 8 APT attack detection results for ACG-BT when replacing SAG with SMOTE

Window (Number of flows before and after APT center)	SMOTE	BiLSTM Layers	Transformers Layers	Evaluation of IP			
				Acc	Pre	Rec	F1
10		1	2	0.8976	0.6262	0.9526	0.7556
10		2	3	0.9268	0.7034	0.9668	0.8144
10		3	4	0.9142	0.6667	0.9668	0.7892

Table 9 APT attack detection results for ACG-BT when replacing SAG with VAE

Window (Number of flows before and after APT center)	VAE	BiLSTM Layers	Transformers Layers	Evaluation of IP			
				Acc	Pre	Rec	F1
10		1	2	0.8806	0.8265	0.9635	0.8897
10		2	3	0.9803	0.9429	0.9384	0.9406
10		3	4	0.9168	0.8857	0.9572	0.9200

Table 10 APT attack detection results of ACG-BT model when replacing BiLSTM + Transformer with CNN + LSTM

Window (Number of flows before and after APT center)	SAG Latten dims	CNN Layers	LSTM Layers	Evaluation of IP			
				Acc	Pre	Rec	F1
10	100	1	2	0.8147	0.7568	0.9275	0.8335
10	100	2	3	0.8481	0.8004	0.9275	0.8593
10	100	3	4	0.8270	0.7726	0.9269	0.8427

Table 11 APT attack detection results of ACG-BT model when replacing BiLSTM + Transformer with BiLSTM + Attention

Window (Number of flows before and after APT center)	SAG Latten dims	BiLSTM Layers	Attention Layers	Evaluation of IP			
				Acc	Pre	Rec	F1
10	100	1	2	0.9417	0.8586	0.7773	0.8159
10	100	2	3	0.9717	0.9108	0.9194	0.9151
10	100	3	4	0.9480	0.8962	0.7773	0.8325

Table 12 APT attack detection results of ACG-BT model when not using AC

SAG Slatted dims	BiLSTM Layers	Transformer Layers	Evaluation of IP			
			Acc	Pre	Rec	F1
100	2	3	0.9324	0.9074	0.8165	0.8592

Table 13 Results of detecting APT attacks of the ACG-BT model without using SAG

Window (Number of flows before and after APT center)	BiLSTM Layers	Transformer Layers	Evaluation of IP			
			Acc	Pre	Rec	F1
10	2	3	0.9501	0.8191	0.7512	0.7837

Table 14 Results of detecting APT attacks of the ACG-BT model without using BT

Window (Number of flows before and after APT center)	SAG Slatted dims	Evaluation of IP			
		Acc	Pre	Rec	F1
10	100	0.8105	0.7552	0.8900	0.8176

Table 15 Features extracted by CICFlowMeter

Features	Descriptions	Data Type
Flow duration	Duration of the flow in Microsecond	Integer
total Fwd Packet	Total packets in the forward direction	Integer
total Bwd packets	Total packets in the backward direction	Integer
total Length of Fwd Packet	Total size of packet in forward direction	Integer
total Length of Bwd Packet	Total size of packet in backward direction	Integer
Fwd Packet Length Min	Minimum size of packet in forward direction	Integer
Fwd Packet Length Max	Maximum size of packet in forward direction	Integer
Fwd Packet Length Mean	Mean size of packet in forward direction	Float
Fwd Packet Length Std	Standard deviation size of packet in forward direction	Float
Bwd Packet Length Min	Minimum size of packet in backward direction	Integer
Bwd Packet Length Max	Maximum size of packet in backward direction	Integer
Bwd Packet Length Mean	Mean size of packet in backward direction	Float
Bwd Packet Length Std	Standard deviation size of packet in backward direction	Float
Flow Byte/s	Number of flow packets per second	Float

Table 15 (continued)

Features	Descriptions	Data Type
Flow Packets/s	Number of flow bytes per second	Float
Flow IAT Mean	Mean time between two packets sent in the flow	Integer
Flow IAT Std	Standard deviation time between two packets sent in the flow	Integer
Flow IAT Max	Maximum time between two packets sent in the flow	Float
Flow IAT Min	Minimum time between two packets sent in the flow	Float
Fwd IAT Min	Minimum time between two packets sent in the forward direction	Integer
Fwd IAT Max	Maximum time between two packets sent in the forward direction	Integer
Fwd IAT Mean	Mean time between two packets sent in the forward direction	Integer
Fwd IAT Std	Standard deviation time between two packets sent in the forward direction	Float
Fwd IAT Total	Total time between two packets sent in the forward direction	Float
Bwd IAT Min	Minimum time between two packets sent in the backward direction	Integer
Bwd IAT Max	Maximum time between two packets sent in the backward direction	Integer
Bwd IAT Mean	Mean time between two packets sent in the backward direction	Integer
Bwd IAT Std	Standard deviation time between two packets sent in the backward direction	Float
Bwd IAT Total	Total time between two packets sent in the backward direction	Float
Fwd PSH flag	Number of times the PSH flag was set in packets travelling in the forward direction (0 for UDP)	Boolean
Bwd PSH Flag	Number of times the PSH flag was set in packets travelling in the backward direction (0 for UDP)	Boolean
Fwd URG Flag	Number of times the URG flag was set in packets travelling in the forward direction (0 for UDP)	Boolean
Bwd URG Flag	Number of times the URG flag was set in packets travelling in the backward direction (0 for UDP)	Boolean
Fwd Header Length	Total bytes used for headers in the forward direction	Integer
Bwd Header Length	Total bytes used for headers in the backward direction	Integer
FWD Packets/s	Number of forward packets per second	Float
Bwd Packets/s	Number of backward packets per second	Float
Min Packet Length	Minimum length of a packet	Integer
Max Packet Length	Maximum length of a packet	Integer
Packet Length Mean	Mean length of a packet	Float
Packet Length Std	Standard deviation length of a packet	Float
Packet Length Variance	Variance length of a packet	Float
FIN Flag Count	Number of packets with FIN	Integer
SYN Flag Count	Number of packets with SYN	Integer
RST Flag Count	Number of packets with RST	Integer
PSH Flag Count	Number of packets with PUSH	Integer
ACK Flag Count	Number of packets with ACK	Integer
URG Flag Count	Number of packets with URG	Integer
CWR Flag Count	Number of packets with CWE	Integer
ECE Flag Count	Number of packets with ECE	Integer
down/Up Ratio	Download and upload ratio	Float
Average Packet Size	Average size of packet	Float
Avg Fwd Segment Size	Average size observed in the forward direction	Float
AVG Bwd Segment Size	Average number of bytes bulk rate in the forward direction	Float
Fwd Header Length	Length of the forward packet header	Integer
Fwd Avg Bytes/Bulk	Average number of bytes bulk rate in the forward direction	Float
Fwd AVG Packet/Bulk	Average number of packets bulk rate in the forward direction	Float
Fwd AVG Bulk Rate	Average number of bulk rate in the forward direction	Float

Table 15 (continued)

Features	Descriptions	Data Type
Bwd Avg Bytes/Bulk	Average number of bytes bulk rate in the backward direction	Float
Bwd AVG Packet/Bulk	Average number of packets bulk rate in the backward direction	Float
Bwd AVG Bulk Rate	Average number of bulk rate in the backward direction	Float
Subflow Fwd Packets	The average number of packets in a sub flow in the forward direction	Integer
Subflow Fwd Bytes	The average number of bytes in a sub flow in the forward direction	Integer
Subflow Bwd Packets	The average number of packets in a sub flow in the backward direction	Integer
Subflow Bwd Bytes	The average number of bytes in a sub flow in the backward direction	Integer
Init_Win_bytes_forward	The total number of bytes sent in initial window in the forward direction	Integer
Init_Win_bytes_backward	The total number of bytes sent in initial window in the backward direction	Integer
Act_data_pkt_forward	Count of packets with at least 1 byte of TCP data payload in the forward direction	Integer
min_seg_size_forward	Minimum segment size observed in the forward direction	Integer
Active Min	Minimum time a flow was active before becoming idle	Float
Active Mean	Mean time a flow was active before becoming idle	Float
Active Max	Maximum time a flow was active before becoming idle	Integer
Active Std	Standard deviation time a flow was active before becoming idle	Integer
Idle Min	Minimum time a flow was idle before becoming active	Float
Idle Mean	Mean time a flow was idle before becoming active	Float
Idle Max	Maximum time a flow was idle before becoming active	Integer
Idle Std	Standard deviation time a flow was idle before becoming active	Float

Author contributions Cho Do Xuan raised the idea, initialized the project, and designed the experiments; Tung and Long carried out the experiments under the supervision of Cho Do Xuan. Both authors analyzed the data and results. Cho Do Xuan and Tung wrote the paper.

Data availability No datasets were generated or analysed during the current study.

Declarations

Competing interests The authors declare no competing interests.

References

- Xuan, C.D., Nguyen, T.T.: A novel approach for APT attack detection based on an advanced computing. *Sci. Rep.* **14**(1), 22223 (2024)
- Singh, S., Sharma, P.K., Moon, S.Y., Moon, D., Park, J.H.: A comprehensive study on APT attacks and countermeasures for future networks and communications: challenges and solutions. *J. Supercomput.* **75**, 4543–4574 (2019)
- Alshamrani, A., Myneni, S., Chowdhary, A., Huang, D.: A survey on advanced persistent threats: techniques, solutions, challenges, and research opportunities. *IEEE Commun. Surveys Tutorials* **21**(2), 1851–1877 (2019)
- Sharma, A., Gupta, B.B., Singh, A.K., Saraswat, V.K.: Advanced persistent threats (apt): evolution, anatomy, attribution and countermeasures. *J. Ambient. Intell. Humaniz. Comput.* **14**(7), 9355–9381 (2023)
- Stojanović, B., Hofer-Schmitz, K., Kleb, U.: APT datasets and attack modeling for automated detection methods: a review. *Comput. Secur.* **92**, 101734 (2020). <https://doi.org/10.1016/j.cose.2020.101734>
- Mamun, M. and Shi, K.: DeepTaskAPT: Insider APT detection using Task-tree based Deep Learning. 2021 IEEE 20th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), pp. 693–700, Shenyang, China, (2021). <https://doi.org/10.1109/TrustCom53373.2021.00102>.
- Niu, W., Zhou, J., Zhao, Y., Zhang, X., Peng, Y., Huang, C.: Uncovering APT malware traffic using deep learning combined with time sequence and association analysis. *Comput. Secur.* **120**, 102809 (2022). <https://doi.org/10.1016/j.cose.2022.102809>
- Han, X., Li, C., Li, X. and Lu, T.: Research on APT attack detection technology based on DenseNet convolutional neural network. 2021 International Conference on Computer Information Science and Artificial Intelligence (CISAI), pp. 440–448, Kunming, China (2021). <https://doi.org/10.1109/CISAI54367.2021.00091>.
- Cho, D.X., Nam, H.H.: A method of monitoring and detecting APT attacks based on unknown domains. *Procedia Comput. Sci.* **150**, 316–323 (2019). <https://doi.org/10.1016/j.procs.2019.02.058>
- Do Xuan, C., Cuong, N.H.: A novel approach for APT attack detection based on feature intelligent extraction and representation learning. *PLoS ONE* **19**(6), e0305618 (2024)
- Yuan, W., Yang, C., Hung Nguyen, Q. V., Cui, L., He, T., Yin, H.: Interaction-level membership inference attack against federated recommender systems. *WWW '23: Proceedings of the ACM Web Conference 2023* Pages 1053–1062 <https://doi.org/10.1145/3543507.3583359>
- Yuan, W., Yuan, S., Yang, C., Quoc Viet Hung, N., Yin, H.: Manipulating visually aware federated recommender systems and its countermeasures. *ACM Trans. Inf. Syst.* **42**(3), 1–26 (2023)

13. Chen, L., Yuan, W., Chen, T., Ye, G., Hung, N.Q., Yin, H.: Adversarial item promotion on visually-aware recommender systems by guided diffusion. *ACM Trans. Inf. Syst.* **42**(6), 1–26 (2024). <https://doi.org/10.1145/3666088>
14. Zongwei, W., Yu, J., Gao, M., Yuan, W., Ye, G., Sadiq, S., Yin, H.: Poisoning Attacks and Defenses in Recommender Systems: A Survey. [arXiv:2406.01022](https://arxiv.org/abs/2406.01022)
15. Yang, C., Yuan, W., Qu, L., Nguyen, T. T.: PDC-FRS: Privacy-preserving Data Contribution for Federated Recommender System. [arXiv:2409.07773](https://arxiv.org/abs/2409.07773)
16. Graph Attention Networks, Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, Yoshua Bengio, To appear at ICLR 2018 <https://doi.org/10.48550/arXiv.1710.10903>
17. Nguyen, H.C., Xuan, C.D., Nguyen, L.T., Nguyen, H.D.: A new framework for APT attack detection based on network traffic. *J. Intell. Fuzzy Syst.* **44**(3), 3459–3474 (2023)
18. Cuong, N.H., Do Xuan, C., Long, V.T., Dat, N.D., Anh, T.Q.: A novel approach for APT detection based on ensemble learning model. *Stat. Anal. Data Min.: ASA Data Sci. J.* **18**(1), e70005 (2025)
19. Xuan, C.D., Huong, D.T., Nguyen, T.: A novel intelligent cognitive computing-based APT malware detection for endpoint systems. *J. Intell. Fuzzy Syst.* **43**(3), 3527–3547 (2022)
20. Panahnejad, M., Mirabi, M.: APT-Dt-KC: advanced persistent threat detection based on kill-chain model. *J. Supercomput.* **78**, 8644–8677 (2022). <https://doi.org/10.1007/s11227-021-04201-9>
21. Xuan, C.D., Duong, D., Dau, H.X.: A multi-layer approach for advanced persistent threat detection using machine learning based on network traffic. *J. Intell. Fuzzy Syst.* **40**(6), 11311–11329 (2021)
22. Xu, B., Gong, Y., Geng, X., et al.: ProcSAGE: an efficient host threat detection method based on graph representation learning. *Cybersecurity* **7**, 51 (2024). <https://doi.org/10.1186/s42400-024-00240-w>
23. Xiang, G., Shi, C., Zhang, Y.: An APT event extraction method based on BERT-BiGRU-CRF for APT attack detection. *Electronics* **12**, 3349 (2023). <https://doi.org/10.3390/electronics12153349>
24. Irshad, H., et al.: TRACE: enterprise-wide provenance tracking for real-time APT detection. *IEEE Trans. Inf. Forensics Secur.* **16**, 4363–4376 (2021). <https://doi.org/10.1109/TIFS.2021.3098977>
25. Saini, N., Bhat Kasaragod, V., Prakasha, K., Das, A.K.: A hybrid ensemble machine learning model for detecting APT attacks based on network behavior anomaly detection. *Concurr. Comput.: Practice Exp.* **35**(28), e7865 (2023)
26. Chen, T., Dong, C., Lv, M., Song, Q., Liu, H., Zhu, T., and Fan, Y.: APT-KGL: An intelligent APT detection system based on threat knowledge and heterogeneous provenance graph learning. *IEEE Transactions on Dependable and Secure Computing* (2022)
27. Do Xuan, C.: Detecting APT attacks based on network traffic using machine learning. *J. Web Eng.* **20**(1), 171–190 (2021)
28. Fawaz, H. I., Forestier, G., Weber, J., Idoumghar, L., and Muller, P. A.: Deep learning for time series classification: a review. *Data Mining and Knowledge Discovery*, 33(4), 917–963. (2019) Link
29. Zhao, G., Liu, P., Sun, K., Yang, Y., Lan, T., Yang, H.: Research on data imbalance in intrusion detection using CGAN. *PLoS ONE* **18**(10), e0291750 (2023)
30. Wu, X., Li, J., Jin, Y., Zheng, S.: Modeling and analysis of tool wear prediction based on SVD and BiLSTM. *Int. J. Adv. Manuf. Technol.* **106**, 4391–4399 (2020)
31. Vaswani, A.: Attention is all you need. *Advances in Neural Information Processing Systems* (2017)
32. Malware Capture Facility Project. Available online: <https://www.stratosphereips.org/datasets-malware>. (accessed on 8 June 2020).
33. Quang Nam Portal. Available online: <http://english.quangnam.gov.vn/default.aspx> (accessed on 8 June 2020)
34. Lin, S., Clark, R., Birke, R., Schönborn, S., Trigoni, N., and Roberts, S.: Anomaly detection for time series using vae- lstm hybrid model. In *ICASSP 2020–2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* (pp. 4322–4326). IEEE (2020)
35. Bao, F., Wu, Y., Li, Z., Li, Y., Liu, L., Chen, G.: Effect improved for high-dimensional and unbalanced data anomaly detection model based on KNN-SMOTE-LSTM. *Complexity* **2020**(1), 9084704 (2020)
36. Al, A. A. A. M. H.: IoT Malware Detection Based On Anomaly Traffic Identification Using CNN-LSTM. In *2024 16th International Conference on Electronics, Computers and Artificial Intelligence (ECAI)* (pp. 1–6). IEEE (2024)
37. Li, X., Niu, W., Zhang, X., Zhang, R., Yu, Z., and Li, Z.: Improving performance of log anomaly detection with semantic and time features based on bilstm-attention. In *2021 2nd International Conference on Electronics, Communications and Information Technology (CECIT)* (pp. 661–666). IEEE (2021)
38. Tong, X., Tan, X., and Sun, X.: Abnormal behavior detection based on GCN-BiLSTM. In *Third International Conference on Machine Learning and Computer Application (ICMLCA 2022)* (Vol. 12636, pp. 468–474). SPIE (2023)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.